

3 Functions and Macros Homework

Sai Ram Vodnala

06-13-2024

General instructions.

There are four exercises below. Three exercises are worth 10 points each, and are labelled (Either R, Python or Julia). For these exercises, you can implement a solution in the language of your choice. I've included code chunk templates for all three languages, but you only need to fill in the chunks for one language.

One exercise is labelled (R, Python and Julia). This exercise is worth 20 points, and you must provide a solution using all three languages. This exercise also includes code chunk templates for all three languages.

Some of the exercises have questions relating to the results. Write your answers in the narrative part of the text, formatted as *italicized* text, **bold** text or

quoted text.

Change the name of this file to match your user name on D2L, keeping the 'Rmd' extension, and include week number in the title (for example, **Peter.Claussen.3.Rmd**). Upload this file to D2L. Typeset this file to Word or PDF and upload the result to D2L as well.

Exercise 1 (Either R, Python or Julia)

As before, using values from Table 1, show how Wansink and Payne calculate percent increase in calories per recipe (43.7%), calories per serving (63.0%) and servings per recipe (-2.0). That is, confirm the values in the final column of Table 1 (Change from 1936 to 2006, %, p. 292).

For this exercise, write a function to perform these calculations. Name the function **PercentIncrease** with parameters **m1** and **m2**. Check that this function duplicates the values you calculated for Homework 2.

Answer

Function definition

R

```
# function definition
Percent.increase <- function(m_1, m_2){

  percent_increase_calories_recipe = ((m_2 -m_1)/m_1)*100

  return(percent_increase_calories_recipe)
}
```

Python

```
# function definition
```

Julia

```
# function definition
```

Calories per Recipe

R

```
print(Percent.increase(2123.8,3051.9))
```

```
## [1] 43.69997
```

Python

Julia

Calories per Serving

R

```
print(Percent.increase(268.1,384.4))
```

```
## [1] 43.37934
```

Python

Julia

Servings per Recipe

R

```
print(Percent.increase(12.9,12.7))
```

```
## [1] -1.550388
```

Python

Julia

Exercise 2 (Either R, Python or Julia)

Consider this quotation from Wansink

Over the past 70 years, the total caloric content increased for 14 of the 18 recipes. Because of changes in ingredients, the mean average calories in a recipe increased by 928.1 (from 2123.8 calories [95% CI, 1638.7 to 2608.9 calories] to 3051.9 calories [CI, 2360.7 to 3743.1 calories]), representing a 43.7% increase ($P < 0.001$). As the Table indicates, mean average calories per serving increased for 17 of 18 recipes and was influenced by both changes in ingredients and changes in serving size. The resulting increase of 168.8 calories (from 268.1 calories [CI, 210.4 to 325.8 calories] to 436.9 calories [CI, 359.1 to 514.7 calories]) represents a 63.0% increase ($P < 0.001$) in calories per serving.

Write a function named `ConfidenceInterval` that returns a **list** or **tuple** (depending on the language) comprised of the **Lower** and **Upper** bounds. **Do not return an array**

You may use the `ConfidenceBound` function from course outline to produce confidence intervals or write your own version. Your function should accept as arguments `mean`, `sd` and `n`, and an optional parameter `alpha=0.05`.

Confirm the confidence intervals reported as

- [95% CI, 1638.7 to 2608.9 calories]
- [CI, 2360.7 to 3743.1 calories]
- [CI, 210.4 to 325.8 calories]
- [CI, 359.1 to 514.7 calories]

Use the appropriate values from Table 1.

Answer

Define your function(s) in the code chunk below, then call the function with appropriate parameters in the following sections

R

```
# function definitions
SE<- function(sd,n){
  sd/sqrt(n)
}

ConfidenceBound <- function(sd, n, alpha=0.05) {
  qnorm(1-alpha/2)*SE(sd,n)
}

ConfidenceInterval_Normal_distribution <- function(mean,sd,n) {
```

```

lower<- (mean - ConfidenceBound(sd,n))
upper<- (mean + ConfidenceBound(sd,n))
return(list(Lower_Ndistribution=lower, Upper_Ndistribution=upper))
}

ConfidenceBoundT <- function(mean,sd,n) {
pt(0.95,n-1,lower.tail = TRUE, log.p = FALSE)

}

ConfidenceInterval_T_distribution <- function(mean,sd,n) {
lower<- (mean - ConfidenceBoundT(mean,sd,n))
upper<- (mean + ConfidenceBoundT(mean,sd,n))
return(list(Lower_tdistribution=lower,Upper_tdistribution =upper))

}

```

Python

```
# function definitions
```

Julia

```
# function definitions
```

[95% CI, 1638.7 to 2608.9 calories]

R

```
ConfidenceInterval_Normal_distribution(2123.8, 1050.0,18)
```

```
## $Lower_Ndistribution
## [1] 1638.734
##
## $Upper_Ndistribution
## [1] 2608.866
```

```
ConfidenceInterval_T_distribution(2123.8, 1050.0, 18)
```

```
## $Lower_tdistribution
## [1] 2122.978
##
## $Upper_tdistribution
## [1] 2124.622
```

Python

Julia

[CI, 2360.7 to 3743.1 calories]

R

```
ConfidenceInterval_Normal_distribution(3051.9, 1496.2,18)
```

```
## $Lower_Ndistribution
## [1] 2360.704
##
## $Upper_Ndistribution
## [1] 3743.096
```

```
ConfidenceInterval_T_distribution(3051.9, 1496.2,18)
```

```
## $Lower_tdistribution
## [1] 3051.078
##
## $Upper_tdistribution
## [1] 3052.722
```

Python

Julia

[CI, 210.4 to 325.8 calories]

Here, I am Assuming Mean of calories per serving value for the year 2006 is 384.4 and Standard Deviation(SD)=168.3 to match the above confidence level. Unable to match the CI when I consider Mean 436.9

R

```
ConfidenceInterval_Normal_distribution(436.9, 168.8,18)
```

```
## $Lower_Ndistribution
## [1] 358.9198
##
## $Upper_Ndistribution
## [1] 514.8802
```

```
ConfidenceInterval_T_distribution(436.9, 168.8,18)
```

```
## $Lower_tdistribution
## [1] 436.0777
##
## $Upper_tdistribution
## [1] 437.7223
```

Python

Julia

[CI, 359.1 to 514.7 calories]

Here, I am Assuming Mean of calories per serving value for the year 2006 is 384.4 and Standard Deviation(SD)=168.3 to match the above confidence level. Unable to match the CI when I consider Mean 436.9

R

```
ConfidenceInterval_Normal_distribution(436.9, 168.3,18)
```

```
## $Lower_Ndistribution
## [1] 359.1508
##
## $Upper_Ndistribution
## [1] 514.6492
```

```
ConfidenceInterval_T_distribution(436.9, 168.3,18)
```

```
## $Lower_tdistribution
## [1] 436.0777
##
## $Upper_tdistribution
## [1] 437.7223
```

Python

Julia

Where you able to reproduce Wansink and Payne's reported confidence intervals?

Exercise 3 (Either R, Python or Julia)

Implement the probability density function for the normal distribution as a function.

$$f(x, \mu, \sigma^2) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

Define μ and σ as optional parameters, taking values `mu=0` and `sigma=1`. Name this function `NormPdf`.

Answer

Define your function(s) in the code chunk below, the call the function with appropriate parameters in the following sections. Use $\mu = 1$, $\sigma = 2$ and $x = \{-1, 0, 1\}$ and compare with previous homework.

R

```
# function definition
mu <- 1
sigma <- 2
x <- c(-1,0,1)
NormPdf <- function(x,mu,sigma){
  (1/(sigma*sqrt(2*pi)))*exp(-((x-mu)^2)/(2*sigma^2))
}
```

Python

```
# function definition
```

Julia

```
# function definition
```

$x = -1$

R

```
print(NormPdf(-1,1,2))
```

```
## [1] 0.1209854
```

Python

Julia

$x = 0$

R

```
print(NormPdf(0,1,2))
```

```
## [1] 0.1760327
```

Python

Julia

$x = 1$

R

```
print(NormPdf(1,1,2))
```

```
## [1] 0.1994711
```

Python

Julia

Exercise 4 (R, Python and Julia)

The probability mass function for the binomial distribution is given by

$$\frac{n!}{k!(n-k)!} \times \pi^k \times (1-\pi)^{(n-k)}$$

where π is the probability of success and k is the number of successes out of n trials. Implement this as a function `BinomPMF` that accepts as arguments `k`, `n` and `pi`. Test your function with values $\pi = 0.5$, $n = 10$ and $k = \{3, 4, 5\}$

Answer

Part a.

Define your function(s) in the code chunk below, then call the function with appropriate arguments in the following sections

R

```
# function definition
pi <- 0.5
n <- 10
k <- c(3, 4, 5)
BinomPMF <- function(k, n, pi) {
  factorial(n) / (factorial(k) * factorial(n - k)) * pi^k * (1 - pi)^(n - k)
}
```

Python


```
# function definition
from math import factorial
pi = 0.5
n=10
def BinomPMF(k, n, pi):
    return factorial(n) / (factorial(k) * factorial(n - k)) * pi**k * (1 - pi)**(n - k)
```

Julia

```
# function definition
function BinomPMF(k, n, pi)
    bpmf = binomial(n, k) * (pi ^ k) * ((1 - pi) ^ (n - k))
    return bpmf
end
```

```
## BinomPMF (generic function with 1 method)
```

```
k = 3
```

R

```
print(BinomPMF(3,10,0.5))
```

```
## [1] 0.1171875
```

Python

```
print(BinomPMF(3,10,0.5))
```

```
## 0.1171875
```

Julia

```
print(BinomPMF(3,10,0.5))
```

```
## 0.1171875
```

```
k = 4
```

R

```
print(BinomPMF(4,10,0.5))
```

```
## [1] 0.2050781
```

Python

```
print(BinomPMF(4,10,0.5))
```

```
## 0.205078125
```

Julia

```
print(BinomPMF(4,10,0.5))
```

```
## 0.205078125
```

$k = 5$

R

```
print(BinomPMF(5,10,0.5))
```

```
## [1] 0.2460938
```

Python

```
print(BinomPMF(5,10,0.5))
```

```
## 0.24609375
```

Julia

```
print(BinomPMF(5,10,0.5))
```

```
## 0.24609375
```

You can check your work against the built in Binomial distribution functions, `dbinom(k,n,pi)` in R, `scipy.stats.norm.pdf(k,n,pi)` in Python and `pdf(Binomial(n, pi),k)` in Julia.

R

```
dbinom(3, 10, 0.5)
dbinom(4, 10, 0.5)
dbinom(5, 10, 0.5)
```

Python

```
import scipy

scipy.stats.binom.pmf(3, 10, 0.5)
scipy.stats.binom.pmf(4, 10, 0.5)
scipy.stats.binom.pmf(5, 10, 0.5)
```

Julia

```
using Distributions

pdf(Binomial(10, 0.5),3)
```

```
## 0.11718750000000004
```

```
pdf(Binomial(10, 0.5),4)
```

```
## 0.205078125000000033
```

```
pdf(Binomial(10, 0.5),5)
```

```
## 0.246093750000000022
```

Part b.

In “An Argument for Divine Providence, Taken from the Constant Regularity Observ’d in the Births of Both Sexes.” published in 1710 (<https://royalsocietypublishing.org/doi/epdf/10.1098/rstl.1710.0011>),

In London in 1629, there were 5218 males born and 4683 females. Assuming male and female births are equally likely, what is the probability of this occurrence happening by chance - that is, calculate the probability of the binomial distribution with $\pi = 0.5$, $k = 5218$ and $n = 5218 + 4683$ using your **BinomPMF** function.

What happens when you try to calculate this value? Hint: what happens when the factorial function is given large values as arguments? You might not be able to typeset code, if it results in an error. Simply report the resulting error messages.

Are there differences among the three languages that you might use to calculate this value?

There is no difference and the calculated value is same in three languages.

```
dbinom(5218, 9901, 0.5)
```

```
import scipy
scipy.stats.binom.pmf(5218, 9901, 0.5)
```

```
using Distributions
binompdf(Binomial( 9901, 0.5), 5218)
```

An interesting side note. Arbuthnot's data is available as an R data set associated with the **HistData** library. According to the R help file for this data set (one version is available at <https://www.stat.auckland.ac.nz/~wild/data/Rdatasets/doc/HistData/Arbuthnot.html>), a publication (S. Zabell (1976). Arbuthnot, Heberden, and the Bills of Mortality. Technical Report No. 40, Department of Statistics, University of Chicago.) points out several inaccuracies or irregularities in these data. Erroneously published data has been around a long time.